

CSED311: Lab 2 RTL Design

컴퓨터공학과 20180806 류누리

컴퓨터공학과 20180285 이신범

Introduction

Finite State Machine (FSM) 어떠한 입력에 의해 다른 상태로 전이하는 방식으로 작동한다. 두가지 형식이 있는데, mealy machine에서는 output이 현재 상태와 input에 의해 결정이 된다. Moor machine에서는 output이 현재 state에 의해서만 결정된다. 자판기에 대해서도 다음과 같이 생각해볼 수 있다. 두 가지 상태가 있는데, 하나는 돈이 들어있는 상태이고, 다른 하나는 돈이 들어오지 않은 상태이다. 돈을 넣으면 돈을 넣은 상태로 전이하고, 돈을 넣은 상태에서는 자판기 사용 시간이 다 지나거나, reset을 누른 경우 다시 돈을 넣지 않은 상태로 전이한다.

이번 랩에서는 vending machine을 verilog를 사용하여 구현하는 것을 목표로 한다. 모든 use-case에 맞게 기능을 구현해야 한다. Vending machine은 사용하는데 sequence가 있으며, waiting time이 지나거나 reset을 누르는 경우 sequence가 초기화된다. 각 sequence에 알맞게 순차적으로 기능을 구현해야 한다.

이번 랩 과제를 통해서 우리는 FSM에 대한 이해도를 높일 수 있을 것이며, 직접 vending machine을 구현함으로써 실용적인 지식을 쌓을 수 있을 것이다.

Design

이번 Project 에서는, sequential logic과 Combinational logic을 나눠서 코드를 작성하였고, 별도의 sub module과 large module을 분리하는 작업을 수행하지는 않았다.

Combinational logic에서는 명령에 따라 연산을 진행하고, sequential logic에서는 clk에 따라서, 1)reset을 진행하거나, 2) total_money에 total_meney_next state를 밀어주는 작업을 진행하였다.

Clk의 변화를 이용해야 하므로, 이 둘의 사용을 명확히 이해하고 있어야 설계가 가능한 machine 이었다고 생각한다.

Implementation

0 우리는 total_money를 통해 현재 입력된 돈의 양을 계산하였다. (계산의 편의를 위해, 단위는 100원으로 나눠서 1원 / 5원 /10원으로 계산하였다)

1 Clk의 변화에 따라 tb file에서 특정 명령을 내리면, 그 명령이 1) 돈을 넣는다 / 2) 물건을 산다 일 경우에, total_money_next에 값을 넣어주었다. 그 뒤 , sequential circuit을 통해 total_money <= total_money_next를 해 주어, 다음 state로 넘어갈 수 있도록 하였다.

2 O_availabe_item의 경우, for문을 사용하는 대신 여러 개의 if문을 두어서, 현재 total_money에 따라 이용가능한 물품을 표현하도록 설정했다.

3 wait_time을 설정하여, 특정 시간 이상동안 동작이 수행되지 않으면, return을 하도록 코드를 작성하였다. 초기 initial state에서 Wait_time = `kWaitTime 으로 설정하고,

‘어떠한 작업이 이루어지면’ Wait_time = `kWaitTime으로 다시 초기화 시켜서 작업이 일어났음을 명시하고,

‘어떠한 작업도 이루어지지 않으면’ Wait_time = Wait_time -1; 을 해주어서, Wait_time == 0이 되면 return이 진행하게했다.

따라서 ReturnTest로 넘어갈 때, 2800원이 된 후 아무런 동작을 하지 않으므로, 일정 시간이 지나면 o_return_coin이 설정되고 return을 진행하도록 하였다. (동전을 어떻게 거슬러주는지는 상관없다는 답변을 받아서, 100원짜리로 모두 거슬러주도록 설정하였다)

또한, i_trigger_return == 1인 경우, 바로 o_return_coin을 1로 설정하여, 100원짜리로 모두 거슬러주도록 코드를 설계하였다.

마지막으로, return 작업도 어떠한 작업이 수행되는 것이므로, 모든 return이 이루어진 뒤 wait_time = kWaitTime으로 다시 설정해주었다.

Discussion

처음에는 자판기에 넣은 돈에 해당하는 state들을 모두 만들 생각을 했지만, 그건 불필요하다고 생각하고, 돈을 넣은 상태와 돈을 넣지 않은 상태로 크게 두 state를 사용해서 구현했다.

Wait_time을 설정하는 것에 차질을 겪었다. 동전을 넣고 기다리는 시간을 구현하는 것에 시간을 제일 많이 쏟은 것 같다.

Transcript 상으로는 pass가 뜨지만, wave 상으로는 아직 처리가 끝나지 않은 test에 대해서 pass가 뜨는 현상이 발생하였고, 이를 수정하였다.

Conclusion

결론적으로, 우리는 introduction에 있는 모든 목표를 달성하는데 성공했으며, 모든 test에서 pass를 얻을 수 있었다. 지시 사항 중 구현하지 못한 것은 없다.